Phytium飞腾

飞腾嵌入式 optee 扩大安全内存

（V1.2）

飞腾信息技术有限公司
www.phytium.com.cn
2023 年 05 月 12 日

版权所有 © 飞腾信息技术有限公司 2023。 保留一切权利。

未经本公司同意，任何单位、公司或个人不得擅自复制、翻译、摘抄本文档内容的部分或全部，不得以任何方式或途径进行传播和宣传。

商标声明

Phytium 和其他飞腾商标均为飞腾信息技术有限公司的商标。

本文档提及的其他所有商标或注册商标，由各自的所有人拥有。

特别提示

本文档仅作为使用指导，飞腾对本文档内容不做任何明示或暗示的声明或保证。本文档中的所有陈述、信息和建议不构成任何明示或暗示的担保。

由于产品版本升级或其他原因，本文档内容会不定期进行更新，如有变更，恕不另行通知。

最新技术资源

飞腾嵌入式软件开源社区： https://gitee.com/phytium_embedded

或者访问飞腾软件开发者平台： https://service.phytium.com.cn/developer/

目录

1 飞腾平台安全侧内存的分配 1
1.1 MMU 1
1.2 E2000D 的内存分配 1
1.3 飞腾 OPTEE 对可用内存的默认配置 1
1.3.1 总体分配 1
1.3.2 系统内存 2
2 扩大 TA 使用的内存 3
2.1 客户需求 3
2.2 安全内存池 3
2.3 注册新的 TA_RAM 3
2.4 TA_RAM 挂接到安全内存池 4
2.5 操作实例 4
2.5.1 操作说明 4
2.5.2 操作和调试过程 5
A 术语和缩略语 9
B 更新记录 10

1 飞腾平台安全侧内存的分配

1.1 MMU

飞腾平台的内存管理一般要启用 MMU（内存管理单元）进行内存管理，在 ARM Trustzone 体系下，分为安全世界和普通世界，MMU 在两个世界中，TLB（页表缓存）是共享的，但 xlat table（转换页表）是独立的。所以安全侧的操作系统 OPTEE 的内存使用遵循安全侧的 xlat table，也就是说安全侧的虚实地址转化是需要独立配置并使能的。

1.2 E2000D 的内存分配

E2000D 参考板的内存为 4G。

其中 2G 映射成 32bit 的低端地址是：0x80000000~0xFFFFFFFF 的区间。

另外 2G 映射成 64bit 的高端地址是：0x2000000000~0x207FFFFFFF 的区间。

目前配置下，通过相关沟通后，划分内存 0xFC000000~0xFFFFFFFF 的 64MB 空间为固件使用，REE 侧操作系统（Linux）不会操作该段内存。

其中 0xFF000000 开始的 16MB 空间为 PBF（飞腾特有固件）专有的空间。OPTEE 可以使用 0xFC000000~0xFEFFFFFF 的 48MB 空间。

1.3 飞腾 OPTEE 对可用内存的默认配置

1.3.1 总体分配

总体的默认配置如下：

```
CFG_TZDRAM_START ?= 0xFC000000
CFG_TZDRAM_SIZE  ?= 0x02200000
CFG_SHMEM_START  ?= 0xFE200000
CFG_SHMEM_SIZE   ?= 0x00C00000
```

从这一段配置可以看出，OPTEE 将 48MB 的内存空间分为了两块：其中一块作为 OPTEE 系统内存，另一块作为和 REE 侧交互用的共享内存。

系统内存从 0xFC000000 开始，大小有 34MB。共享内存从 0xFE2000000 开始，大小为 14MB。

1.3.2 系统内存

OPTEE 对能使用的系统内存，根据功能又细分成了三块。

1) 代码段只读内存 TEE_RAM_RX。

2) 内核使用内存 TEE_RAM_RW。

3) TA 使用内存 TA_RAM。

其中 TEE_RAM_RX+TEE_RAM_RW 的大小总共是 4MB，具体分配情况是按照 tee.bin 生成后代码段的长度来分配的，可以反汇编 tee.bin 后查询代码段大小差不多是 500KB 左右，系统初始化时会根据实际情况分配一个 600KB 的空间给 TEE_RAM_RX，4MB 的剩余空间则给 TEE_RAM_RW。对于 OPTEE 这个轻量级的 OS，空间已经够用了。

分配较多的是 TA_RAM，系统默认给了 30MB，用于加载 TA，以及 TA 运行时使用的安全内存。

2 扩大 TA 使用的内存

2.1 客户需求

OPTEE 作为轻量级的操作系统，TEE_RAM_RX+TEE_RAM_RW 达到 4MB，已经可以应付正常的运行。如果后续有大量的驱动要运行，或扩充 syscall 和库的支持时，才需要扩大这部分内存。

根据之前的交流，很多客户基于 OPTEE 开发 TA 程序，而某些 TA 程序在运行过程中需要大量安全内存，目前默认分配的 30MB 空间远远不够。

本节主要针对怎么扩大 TA_RAM 进行说明。

2.2 安全内存池

OPTEE 对于 TA_RAM 是采用内存池的方式管理，TA_RAM 的内存池对应的全局变量是 tee_mm_sec_ddr。通过完成一个小工具 pool stats（相关源码开源在 Gitee.com 的 Phytium-optee 仓库中），可以查询当前状态下 OPTEE 中 TA_RAM 内存池的大小。如下：

```
root@E2000-Ubuntu:~# pool_stats
D/TC:3 0 core_mmu_xlat_table_alloc:526 xlat tables used 3 / 5
D/TC:? 0 tee_ta_init_pseudo_ta_session:296 Lookup pseudo TA d96a5b40-e2c7-b1af-8794-1002a5d5c61b
D/TC:? 0 tee_ta_init_pseudo_ta_session:309 Open stats.ta
D/TC:? 0 tee_ta_init_pseudo_ta_session:326 stats.ta : d96a5b40-e2c7-b1af-8794-1002a5d5c61b
F/TC:? 0 plat_prng_add_jitter_entropy:72 0xFEEF
Name:Heap
Size:0x30eb0
A/M :0x26c0/0x26e0
Name:Secure DDR
Size:0x1e00000
A/M :0x0/0x0
D/TC:? 0 tee_ta_close_session:529 csess 0xfc0dca40 id 1
D/TC:? 0 tee_ta_close_session:548 Destroy session
```

可以查看到默认的 TA_RAM 是 30MB。本节的目标就是要扩大这个参数的大小。

2.3 注册新的 TA_RAM

OPTEE 作为一个成熟的操作系统，已经有一套比较完善的内存管理机制，内核对 memory 的管理，细分了很多不同的类型，具体类型参考如下结构定义：

```
enum teecore_memtypes {
    MEM_AREA_END = 0,
    MEM_AREA_TEE_RAM,
    MEM_AREA_TEE_RAM_RX,
    MEM_AREA_TEE_RAM_RO,
    MEM_AREA_TEE_RAM_RW,
    MEM_AREA_INIT_RAM_RO,
    MEM_AREA_INIT_RAM_RX,
    MEM_AREA_NEX_RAM_RO,
    MEM_AREA_NEX_RAM_RW,
    MEM_AREA_TEE_COHERENT,
    MEM_AREA_TEE_ASAN,
    MEM_AREA_IDENTITY_MAP_RX,
    MEM_AREA_TA_RAM,
    MEM_AREA_NSEC_SHM,
    MEM_AREA_RAM_NSEC,
    MEM_AREA_RAM_SEC,
    MEM_AREA_IO_NSEC,
    MEM_AREA_IO_SEC,
```

所以，注册新的 TA_RAM，就是将一块新的内存，向系统注册为 MEM_AREA_TA_RAM 的属性。

可以通过 register_phys_mem_pgdir 函数可以向 OPTEE 系统进行内存的注册。

通过以下代码就可以进行注册了：

```
register_phys_mem_pgdir(MEM_AREA_TA_RAM, base address, size);
```

2.4 TA_RAM 挂接到安全内存池

安全内存池作为 OPTEE 的一种 runtime services，在 init_tee_runtime 函数中进行初始化。该函数最先调用了 core_mmu_init_ta_ram 函数。

在函数 core_mmu_init_ta_ram 中，先提取属性为 TA_RAM 的已注册内存空间，代码如下：

```
core_mmu_get_mem_by_type(MEM_AREA_TA_RAM, &s, &e);
```

然后通过一些检查，最终加入到全局对象 tee_mm_sec_ddr 所管理的结构中。代码如下：

```
tee_mm_final(&tee_mm_sec_ddr);
tee_mm_init(&tee_mm_sec_ddr, ps, size, CORE_MMU_USER_CODE_SHIFT,
            TEE_MM_POOL_NO_FLAGS);
```

2.5 操作实例

2.5.1 操作说明

本次测试没有涉及的 REE 侧的修改，所以仅仅是一种验证测试，在实际应用过程中，TEE 侧开发人员应当与 REE 侧协商好内存的使用，比如哪段高端内存地址空间给 TEE 侧使用，然后可以通过飞腾平台的 TZC（Trustzone Controller）把相应内存设置为安全内存，这样 REE 侧就无法访问该段内存空间，

起到硬件隔离的作用，后续 TEE 侧 OPTEE 操作系统的设置和操作才是真正安全的，否则还是有可能出现内存操作冲突的情况。

本次测试操作存在内存操作冲突的可能性。

2.5.2 操作和调试过程

第一步：注册新的内存为 TA_RAM

如下图所示：

```
register_phys_mem_pgdir(MEM_AREA_TA_RAM,DRAM2_BASE, CORE_MMU_PGDIR_SIZE);
```

可以在需要编译的文件中加入，建议加在 BSP 目录的 main.c 文件中。

第二步：配置内存属性

配置增加内存为安全属性，修改如下：

```
--- a/optee_os/core/mm/core_mmu.c
+++ b/optee_os/core/mm/core_mmu.c
@@ -34,6 +34,7 @@
 #endif

 #define SHM_VASPACE_SIZE       (1024 * 1024 * 32)
+#define TEST_BASE              0x2000000000UL

 /*
  * These variables are initialized before .bss is cleared. To avoid
@@ -66,6 +67,7 @@ static struct memaccess_area secure_only[] __nex_data = {
        MEMACCESS_AREA(TRUSTED_SRAM_BASE, TRUSTED_SRAM_SIZE),
 #endif
        MEMACCESS_AREA(TRUSTED_DRAM_BASE, TRUSTED_DRAM_SIZE),
+       MEMACCESS_AREA(TEST_BASE, CORE_MMU_PGDIR_SIZE),
 };
```

修改目标内存的范围，防止和其他范围重合，修改如下：

```
register_phys_mem_pgdir(MEM_AREA_TA_RAM,DRAM2_BASE, CORE_MMU_PGDIR_SIZE);

#ifdef DRAM1_BASE
register_ddr(DRAM1_BASE,DRAM1_SIZE);
#endif

#ifdef DRAM2_BASE
register_ddr((DRAM2_BASE + CORE_MMU_PGDIR_SIZE), (DRAM2_SIZE - CORE_MMU_PGDIR_SIZE));
#endif
```

第三步：编译运行

打开 OPTEE 的调试信息开关，编译运行后可以看到注册的新内存加入到结构 static_memory_map 的打印信息中了，如下图：

```
D/TC:0   dump_mmap_table:802 type TA_RAM       va 0xf5e00000..0xf5ffffff pa 0x2000000000..0x20001fffff size 0x00200000 (pgdir)
D/TC:0   dump_mmap_table:802 type NSEC_SHM     va 0xf6000000..0xf6bfffff pa 0xfe200000..0xfedfffff size 0x00c00000 (pgdir)
D/TC:0   dump_mmap_table:802 type TA_RAM       va 0xf6e00000..0xf8bfffff pa 0xfc400000..0xfe1fffff size 0x01e00000 (pgdir)
D/TC:0   dump_mmap_table:802 type IO_SEC       va 0xf8e00000..0xf8ffffff pa 0x30800000..0x309fffff size 0x00200000 (pgdir)
D/TC:0   dump_mmap_table:802 type IO_SEC       va 0xf9000000..0xf91fffff pa 0x28000000..0x281fffff size 0x00200000 (pgdir)
D/TC:0   dump_mmap_table:802 type RES_VASPACE  va 0xf9200000..0xf9bfffff pa 0x00000000..0x009fffff size 0x00a00000 (pgdir)
D/TC:0   dump_mmap_table:802 type SHM_VASPACE  va 0xf9e00000..0xfbdfffff pa 0x00000000..0x01ffffff size 0x02000000 (pgdir)
D/TC:0   dump_mmap_table:802 type TEE_RAM_RX   va 0xfc000000..0xfc09dfff pa 0xfc000000..0xfc09dfff size 0x0009e000 (smallpg)
D/TC:0   dump_mmap_table:802 type TEE_RAM_RW   va 0xfc09e000..0xfc3fffff pa 0xfc09e000..0xfc3fffff size 0x00362000 (smallpg)
```

系统给这段内存分配了对应的虚拟地址 0xf5e00000~0xf5ffffff。

第四步：扩大内存池 size

pool stats 显示的安全内存池大小为 2MB，很明显是两块 TA_RAM 并没有动态叠加起来。从代码上理解，如下图：

```
for (map = get_memory_map(); !core_mmap_is_end_of_table(map); map++)
    if (map->type == type)
        return map;
```

可以看到，系统在初始化内存池时，并没有建立链表等机制，而是直接将 static_memory_map 表中的第一个 TA_RAM 属性的内存空间初始化系统的安全内存池。也就是说，TA_RAM 只能是一个连续的安全内存空间。

这样，那就只能将高端地址的 TA_RAM 扩大，舍弃低端地址的 TA_RAM，测试将 TA_RAM 扩大到 512MB，只需要将原来的 CORE_MMU_PGDIR_SIZE，修改为(CORE_MMU_PGDIR_SIZE*256)即可，注意 secure_only 中申明的空间也需要相应扩大，测试结果如下：

```
root@E2000-Ubuntu:/data# pool_stats
Name:Heap
Size:0x30eb0
A/M :0x26c0/0x26e0
Name:Secure DDR
Size:0x20000000
A/M :0x0/0x0
root@E2000-Ubuntu:/data# hello_world
Invoking TA to increment 42
TA incremented value to 43
root@E2000-Ubuntu:/data# pool_stats
Name:Heap
Size:0x30eb0
A/M :0x2750/0x3df0
Name:Secure DDR
Size:0x20000000
A/M :0x0/0x2d000
```

第五步：回收低端的 TA_RAM

按以上步骤，TA_RAM 已经改用高端内存空间的 512MB 了，低端地址本来那默认的 30MB 空间已经没什么意义了，可以考虑转给 TEE Core 的内存使用，属性是 TEE_RAM。系统配置说明如下：

```
TEE RAM layout without CFG_WITH_PAGER

 +-----------------------------------+ <-- CFG_TZDRAM_START
 | TEE core secure RAM (TEE_RAM)     |
 +-----------------------------------+
 | Trusted Application RAM (TA_RAM)  |
 +-----------------------------------+
 | SDP test memory (optional)        |
 +-----------------------------------+ <-- CFG_TZDRAM_START + CFG_TZDRAM_SIZE

 +-----------------------------------+ <-- CFG_SHMEM_START
 | Non-secure static SHM             |
 +-----------------------------------+ <-- CFG_SHMEM_START + CFG_SHMEM_SIZE
```

可以看出系统已经有一套比较完整的计算方法，互相关联。最简单的修改方法就是直接配置 CFG_TEE_RAM_VA_SIZE 这个配置项，直接修改为 32MB，这样默认的 TA_RAM 就自动变成了只有 2MB（一个 pgdir 页）大小了。

在配置文件中修改该配置项后测试，可以看到 TEE_RAM 扩大了，如下：

```
D/TC:0   dump_mmap_table:802 type TA_RAM       va 0xd7e00000..0xf7dfffff pa 0x2000000000..0x201fffffff size 0x20000000 (pgdir)
D/TC:0   dump_mmap_table:802 type NSEC_SHM     va 0xf8000000..0xf8bfffff pa 0xfe200000..0xfedfffff size 0x00c00000 (pgdir)
D/TC:0   dump_mmap_table:802 type IO_SEC       va 0xf8e00000..0xf8ffffff pa 0x30800000..0x309fffff size 0x00200000 (pgdir)
D/TC:0   dump_mmap_table:802 type IO_SEC       va 0xf9000000..0xf91fffff pa 0x28000000..0x281fffff size 0x00200000 (pgdir)
D/TC:0   dump_mmap_table:802 type RES_VASPACE  va 0xf9200000..0xf9bfffff pa 0x00000000..0x009fffff size 0x00a00000 (pgdir)
D/TC:0   dump_mmap_table:802 type SHM_VASPACE  va 0xf9e00000..0xfbdfffff pa 0x00000000..0x01ffffff size 0x02000000 (pgdir)
D/TC:0   dump_mmap_table:802 type TEE_RAM_RX   va 0xfc000000..0xfc09dfff pa 0xfc000000..0xfc09dfff size 0x0009e000 (smallpg)
D/TC:0   dump_mmap_table:802 type TEE_RAM_RW   va 0xfc09e000..0xfdffffff pa 0xfc09e000..0xfdffffff size 0x01f62000 (smallpg)
```

如图红色划线部分，新增的内存空间加到了 TEE_RAM_RW 中。

这时候默认的 TA_RAM 已经缩小到 2MB，可以考虑不注册该默认空间为 TA_RAM，以保证 static_memory_map 表中只有一个 TA_RAM。

可以将 core_mmu.c 源码中对应注册 MEM_AREA_TA_RAM 函数屏蔽掉。

A 术语和缩略语

名称	全称	解释
OPTEE	Open Portable TEE	一个开源的、可信执行环境运行软件实现的项目，包括操作系统、运行在非安全执行环境中的交互驱动、支持程序和库。
TEE	Trusted Execution Environment	可信的执行环境，在 ARM 的安全架构中，属于安全世界的执行环境。
REE	Rich Execution Environment	丰富的执行环境，在 ARM 的安全架构中，属于非安全世界的执行环境。
TA	Trusted Application	运行在安全世界的应用程序。
CA	Client Application	运行在非安全世界的应用程序，负责和安全世界的应用程序进行交互。

B 更新记录

更新日期	版本号	编制人	说明
2023-04-20	1.0	邓强	首次发布。
2023-05-06	1.1	邓强	采用新模板，优化字体。
2023-05-12	1.2	邓强	增加特别提示，增加缩略语附录，优化目录结构。